RUCHE · LEÇON

Leçon 1 — Les comptes

Généré le 12/10/2026

Pour entrer dans un club, il faut s'inscrire, prouver que l'adresse donnée est bien la sienne, puis présenter sa carte de membre à chaque visite. C'est exactement ce que fait cette tranche : inscription, vérification de l'adresse e-mail, connexion, et les droits du membre sur ses données (les télécharger, tout supprimer).

C'est aussi la tranche la plus sensible côté sécurité : un compte, c'est un mot de passe à protéger et une identité à ne pas laisser voler. Une bonne partie de cette leçon explique les protections, et une vraie faille trouvée pendant la relecture.

---

1. Le parcours d'un membre, de bout en bout

1. Léa remplit le formulaire de l'appli : prénom, e-mail, mot de passe d'au moins 12 caractères.
2. L'appli vérifie le formulaire elle-même, puis l'envoie à l'API (`POST /api/auth/sign-up/email`).
3. L'API crée le compte, **non vérifié**, et envoie un e-mail en français avec un lien.
4. Léa clique sur le lien. L'API marque l'adresse comme vérifiée, ouvre une session, et la renvoie vers la page « Adresse vérifiée » de l'appli.
5. Désormais, chaque requête de Léa porte sa **session** : l'API sait qui parle.
6. Depuis « Mon compte », Léa peut télécharger ses données ou supprimer son compte.

> **Analogie**
>
> La session, c'est le bracelet d'un festival. On le reçoit une fois à l'entrée après avoir montré son billet (le mot de passe), puis on le montre à chaque passage sans ressortir son billet.

---

2. Ne pas forger sa propre serrure

Stocker des mots de passe, fabriquer des jetons de session, résister aux attaques : c'est un métier. Plutôt que de l'écrire nous-mêmes, on utilise **Better Auth**, une bibliothèque éprouvée. Notre travail consiste à la régler correctement.

> **Analogie**
>
> On n'usine pas soi-même la serrure de sa porte d'entrée : on achète une serrure certifiée, et on s'applique à bien la poser.

Le cœur du réglage :

apps/api/src/auth/auth.ts (extrait)

```
emailAndPassword: {
  enabled: true,
  minPasswordLength: PASSWORD_MIN_LENGTH,
  maxPasswordLength: PASSWORD_MAX_LENGTH,
  // Pas de connexion tant que l'adresse n'est pas vérifiée. Effet secondaire voulu :
  // une inscription avec une adresse existante répond comme une inscription normale.
  requireEmailVerification: true,
  // … et c'est le propriétaire de l'adresse qui est prévenu, par e-mail.
  onExistingUserSignUp: async ({ user }) => {
    await mailer.send({
      to: user.email,
      ...existingAccountEmail(languageOf(user), {
        name: user.name,
        signInUrl: `${appUrl}/sign-in`,
      }),
    })
  },
},
```

Remarque : `PASSWORD_MIN_LENGTH` vient de `packages/contracts`. L'appli et l'API lisent la même valeur : impossible que l'une accepte 10 caractères et l'autre en exige 12.

---

3. La base de données : un tableur très strict

Les données vivent dans **PostgreSQL**. On décrit les tables en TypeScript avec **Drizzle**, qui traduit ensuite nos requêtes en SQL.

packages/db/src/schema/auth.ts (extrait)

```
export const user = pgTable('user', {
  id: text('id').primaryKey(),
  name: text('name').notNull(),
  email: text('email').notNull().unique(), // unique : deux comptes ne peuvent pas
partager une adresse
  emailVerified: boolean('email_verified').default(false).notNull(),
  // …
  // Ajouté par Ruche : langue des e-mails et notifications ('fr' ou 'en').
  language: text('language').default('fr').notNull(),
})
```

Le mot de passe n'est pas dans cette table, et surtout il n'est **jamais** stocké tel quel : on garde seulement son **empreinte** (un « hash »). À la connexion, on recalcule l'empreinte du mot de passe tapé et on compare.

> **Analogie**
>
> L'empreinte digitale permet de vérifier que c'est bien toi, mais on ne peut pas reconstruire ta main à partir d'elle. Si la base était volée, les mots de passe ne le seraient pas.

Quand la structure de la base change, drizzle-kit écrit une **migration** : un fichier SQL numéroté (`packages/db/migrations/0000_auth.sql`), appliqué automatiquement au démarrage. C'est le plan de rénovation de la maison, étape par étape, que chaque serveur suit dans l'ordre.

> **À retenir**
>
> Les tests utilisent **PGlite** : un vrai PostgreSQL qui tourne en mémoire, dans le test lui-même. Chaque test part d'une base vide, sans rien installer. Le développement local l'utilise aussi.

---

4. Des e-mails dans la bonne langue… et désamorcés

Le membre reçoit ses e-mails dans sa langue, enregistrée à l'inscription. Les textes viennent du même paquet de traductions que l'appli.

Un détail compte beaucoup : le prénom du membre est inséré dans du HTML. Si quelqu'un s'inscrivait avec le « prénom » `<img src=x onerror=...>`, ce code finirait dans l'e-mail. On **échappe** donc tout ce qui vient d'un utilisateur :

apps/api/src/email/templates.ts

```ts
const escapeHtml = (value: string) =>
  value
    .replaceAll('&', '&amp;')
    .replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;')
    .replaceAll('"', '&quot;')
    .replaceAll("'", '&#39;')
```

> **Analogie**
>
> Échapper, c'est mettre des guillemets autour de ce que dit quelqu'un. « Ouvrez la porte » écrit entre guillemets dans un compte rendu n'ouvre aucune porte : c'est une citation, plus un ordre.

5. Ne pas révéler qui est inscrit

Si l'API répondait « cette adresse a déjà un compte », n'importe qui pourrait tester des adresses et savoir qui utilise Ruche. Alors elle répond **exactement pareil** dans les deux cas, et prévient discrètement le vrai propriétaire de l'adresse par e-mail (voir `onExistingUserSignUp` plus haut).

Le test qui le garantit compare les deux réponses :

apps/api/src/accounts.test.ts (extrait)

```
it('answers exactly like a new sign-up, without revealing that the address exists',
async () => {
  const first = await ctx.signUp(lea)
  const second = await ctx.signUp({ ...lea, name: 'Someone else', password: 'another-
password-1' })

  expect(second.status).toBe(first.status)
  // … et les deux réponses ont exactement les mêmes champs
})
```

> **Analogie**
>
> Le portier d'un immeuble répond toujours « je transmets », que la personne habite là ou non. Puis il prévient l'habitant, s'il existe.

6. Cinq essais, pas un de plus

Un robot peut essayer des milliers de mots de passe par minute. La **limitation de débit** compte les tentatives par adresse IP, et bloque au-delà de la limite :

apps/api/src/auth/auth.ts (extrait)

```
customRules: {
  '/sign-in/email': { window: 15 * 60, max: 5 },
  '/sign-up/email': { window: 60 * 60, max: 10 },
  '/send-verification-email': { window: 60 * 60, max: 5 },
  '/delete-user': { window: 15 * 60, max: 5 },
},
```

apps/api/src/accounts.test.ts

```ts
it('blocks further attempts after 5 failures in 15 minutes', async () => {
  await signUpAndVerify()

  for (let attempt = 1; attempt <= 5; attempt++) {
    const res = await ctx.signIn({ ...lea, password: `wrong-password-${attempt}` })
    expect(res.status).toBe(401)
  }
  // Même avec le bon mot de passe, la 6e tentative est bloquée.
  const blocked = await ctx.signIn(lea)

  expect(blocked.status).toBe(429)
})
```

Analogie

Le digicode qui se bloque un quart d'heure après cinq mauvais codes.

Piège évité

Pour compter par adresse IP, il faut connaître la vraie adresse du visiteur. Derrière Caddy, l'API ne voit que Caddy. Caddy ajoute donc l'en-tête `X-Real-IP` en **écrasant** toute valeur envoyée par le visiteur : sinon, un robot pourrait s'inventer une nouvelle adresse à chaque essai.

---

7. Comment l'API sait qui parle : le middleware de session

Pour chaque requête vers nos routes `/v1/…`, un middleware lit le cookie (ou le jeton de l'appli mobile), demande à Better Auth à quelle session il correspond, et range le membre dans la requête. Les routes n'ont plus qu'à regarder :

apps/api/src/routes/me.ts

```ts
app.openapi(meRoute, (c) => {
  const user = c.get('user')
  if (!user) return c.json(errorBody(languageOf(c), 'unauthorized'), 401)
  return c.json(toMe(user), 200)
})
```

Le code `401` veut dire « je ne sais pas qui tu es » : l'appli propose alors de se connecter.

8. Mes données : les télécharger, tout effacer

Le RGPD donne à chacun le droit de récupérer ses données et de les faire effacer.

Pour l'export, on choisit les colonnes **une par une**. Ainsi, impossible d'exporter par erreur l'empreinte du mot de passe ou un jeton de session, puisqu'on ne les demande jamais :

apps/api/src/routes/me.ts (extrait)

```
const loginMethods = await db
  .select({ provider: schema.account.providerId, createdAt: schema.account.createdAt })
  .from(schema.account)
  .where(eq(schema.account.userId, user.id))
```

Pour la suppression, la base fait le ménage elle-même grâce à la **suppression en cascade** : chaque session et chaque méthode de connexion est attachée au membre avec `onDelete: 'cascade'`. Supprimer le membre supprime tout ce qui lui est attaché.

9. Côté appli : un formulaire qui vérifie avant d'envoyer

L'écran d'inscription garde ce que tape le membre dans des **états** (`useState`), vérifie le formulaire avec le même schéma Zod que l'API, puis appelle l'API :

apps/app/src/screens/SignUpScreen.tsx

```
const submit = async () => {
  // 1. Vérification locale avec le même schéma que l'API (packages/contracts).
  const parsed = SignUpRequest.safeParse({ name, email, password, language:
i18n.language })
  if (!parsed.success) {
    setFieldErrors(fieldErrorsFrom(parsed.error))
    return
  }
  setFieldErrors({})
  setError(null)
  setLoading(true)
  // 2. Envoi à l'API, puis 3. écran suivant ou message d'erreur.
  const result = await signUp(parsed.data)
  setLoading(false)
  if (result.ok) {
    router.push({ pathname: '/check-email', params: { email: parsed.data.email } })
  } else {
    setError(result.error)
  }
}
```

> **À retenir**
>
> La vérification dans l'appli sert le confort (le membre voit l'erreur tout de suite). La sécurité, elle, est assurée par le serveur, qui revérifie tout : un pirate n'utilise pas notre appli.

Les écrans ne parlent jamais directement à Better Auth : ils passent par `src/accounts/api.ts`. Dans les tests, on remplace ce fichier par de faux appels (`jest.mock`) et on choisit la réponse :

apps/app/src/screens/SignUpScreen.test.tsx (extrait)

```
jest.mock('../accounts/api', () => ({ signUp: jest.fn() }))

it('tells the visitor to wait after too many attempts', async () => {
  signUpMock.mockResolvedValue({ ok: false, error: 'tooManyAttempts' })
  await renderWithI18n(<SignUpScreen />)

  await fillForm()

  expect(screen.getByText('Trop de tentatives. Réessaie dans quelques
minutes.')).toBeTruthy()
})
```

> **Analogie**
>
> Le mock, c'est le figurant qui donne la réplique pendant une répétition : on peut répéter la scène « l'API refuse » sans attendre que l'API refuse vraiment.

---

10. La faille trouvée pendant la relecture

Avant de rendre la tranche, on relit tout avec la liste de contrôle sécurité. Une question : le lien de l'e-mail peut-il renvoyer le membre vers **un autre site** ? Si oui, un pirate pourrait déclencher un e-mail authentique de Ruche qui mène vers un faux site : c'est une « redirection ouverte », très utile pour l'hameçonnage.

On a écrit le test d'abord… et il a échoué : l'API acceptait le lien piégé. En creusant, on a découvert que Better Auth **coupe ses contrôles d'origine quand il détecte un test**. Nos tests passaient donc avec une serrure désactivée. La correction tient en deux lignes :

apps/api/src/auth/auth.ts (extrait)

```
// Contrôles d'origine toujours actifs, y compris pendant les tests (Better Auth les coupe
// par défaut quand il détecte un test). Ils refusent notamment un lien de retour
// (callbackURL) vers un site extérieur, et les requêtes venues d'un autre site.
disableOriginCheck: false,
disableCSRFCheck: false,
```

Et le test qui la verrouille pour toujours :

apps/api/src/accounts.test.ts

```
it('refuses a verification link that would send the member to another website', async () => {
  // Sinon, un e-mail authentique de Ruche pourrait mener vers un faux site (« open redirect »).
  const res = await ctx.request('/api/auth/sign-up/email', {
    method: 'POST',
    headers: { 'content-type': 'application/json' },
    body: JSON.stringify({ ...lea, callbackURL: 'https://evil.example/phishing' }),
  })

  expect(res.ok).toBe(false)
  expect(ctx.mailer.sent).toHaveLength(0)
})
```

La vraie leçon

Un test vert ne prouve rien s'il ne teste pas la vraie configuration. Pour une protection, on écrit toujours le test qui **tente l'attaque** et vérifie qu'elle échoue.

En production, ces contrôles étaient bien actifs : la faille n'aurait existé que si l'on avait désactivé les contrôles ailleurs. Mais sans ce test, on ne l'aurait jamais su.

Les notions de cette leçon

Notion	En une phrase	Où la voir
Session	Le bracelet qui prouve qui tu es à chaque requête	`apps/api/src/auth/session.ts`
Empreinte (hash)	Ce qu'on garde d'un mot de passe, sans pouvoir le retrouver	table `account`
Migration	Une étape numérotée de l'évolution de la base	`packages/db/migrations`

Notion	En une phrase	Où la voir
Clé étrangère, cascade	Un lien entre tables, et le ménage automatique à la suppression	`schema/auth.ts`
Échappement	Transformer un texte pour qu'il ne puisse pas devenir du code	`email/templates.ts`
Énumération	Deviner qui est inscrit d'après les réponses de l'API	`onExistingUserSignUp`
Limitation de débit	Bloquer après trop de tentatives	`rateLimit`
CORS, CSRF, redirection ouverte	Les attaques qui passent par un autre site	`app.ts`, `auth.ts`
État (`useState`)	Une valeur qui change et redessine l'écran	`SignUpScreen.tsx`
Mock	Un faux module pour tester un écran seul	`*.test.tsx`

Exercices

1. Vivre le parcours. Lance l'API (`pnpm --filter @ruche/api dev`) et l'appli web (`pnpm --filter @ruche/app web`). Crée un compte, cherche le lien de vérification dans le terminal de l'API, ouvre-le, puis va dans « Mon compte » et télécharge tes données. Ouvre le fichier : retrouves-tu une trace de ton mot de passe ? (Tu ne devrais pas.)

2. Les tests comme garde-fous. Dans `packages/contracts/src/accounts.ts`, passe `PASSWORD_MIN_LENGTH` de 12 à 10, puis lance `pnpm test`. Note quels tests échouent et pourquoi : ils protègent une décision. Annule ensuite avec `git checkout packages/contracts`.